Portfolio – Expérience projet

Valentin De Araujo

Sommaire

01 Odomo — Station domotique en Java 2
02 Biosphere7 — Jeu de plateau & IA 3
03 Installation & administration de postes 4
04 Musicalertes — Création d'une base de données 5
05 Installation de services réseaux (Nextcloud) 6
06 Environnement réseau & administration système 7
07 Les Mystères de BUTopia — App C# POO 8
08 Retro-PC Dépannage — Refactorisation technique 9
09 Pixel War — Application JavaScript moderne 10
10 Coloration de graphes — Application Java 11
11 Blogue statique — Script Bash 12
12 Risk prevention in Nouvelle-Aquitaine 13

Odomo — Station domotique en Java

Programmation impérative · Semestre 1 · Solo

Cahier des charges

- Développer une station domotique simulée en Java sous NetBeans.
- Gérer différents capteurs et actionneurs d'une maison (lumières, température, alarme…).
- Utiliser des **tableaux, matrices et boucles** comme seules structures de données.
- Concevoir des **tests unitaires JUnit** pour valider chaque fonctionnalité.

Résultats obtenus

- Station fonctionnelle gérant plusieurs pièces simultanément.
- Tous les tests unitaires validés.
- Code structuré en méthodes claires, respectant les contraintes imposées (tableaux uniquement).

Programme complet — tous les tests JUnit au vert

Méthodes et technologies

Java, NetBeans IDE, JUnit 5, Tableaux 2D, Boucles imbriquées, Méthodes statiques

Compétences BUT exercées

- Réaliser (programmation Java impérative)
- Optimiser (algorithmes sur tableaux et boucles)
- Vérifier / Tester (JUnit)

Commentaires

Apprentissage des bases de la logique algorithmique et de la syntaxe Java. C'est une fondation indispensable pour aborder sereinement la programmation orientée objet par la suite.

Biosphere7 — Jeu de plateau & IA

Algorithmique avancée · Semestre 1 · Solo (phase 1) & Binôme (phase 2)

Cahier des charges

- Développer un jeu de plateau sur une grille 14×14 simulant la compétition entre deux consortiums pour la gestion de plantes dans une biosphère martienne.
- **Phase 1** (individuelle) : implémenter jusqu'à 16 niveaux de règles progressifs débloquant les suivants via l'interface fournie.
- **Phase 2** (binôme) : coder deux intelligences artificielles en compétition sur le niveau final, avec analyse comparative des stratégies.

Résultats obtenus

- 11 niveaux sur 16 validés avec succès.
- Deux IA aux stratégies distinctes codées (greedy vs heuristique de propagation).
- Interface graphique fournie entièrement exploitable.

11 niveaux + 2 IA complètes — Rapport stratégique rédigé

Méthodes et technologies

Java, Matrices 2D, Algorithmes gloutons, Heuristiques IA, Récursivité, Binôme / Git

Compétences BUT exercées

- Réaliser (algorithmique en Java)
- Optimiser (stratégies et heuristiques d'IA)
- Collaborer (binôme)

Commentaires

Découverte de la modélisation de problèmes complexes et de la création d'heuristiques. Savoir optimiser un algorithme sera un atout majeur pour développer des applications backend performantes.

Installation & administration de postes

Systèmes d'exploitation · Semestre 1 · Binôme

Cahier des charges

- Simuler l'installation et la configuration d'un poste informatique via **VMWare Workstation** à partir d'un fichier ISO.
- Installer l'OS **Debian 12** sur une machine virtuelle aux ressources très limitées (20 Go d'espace alloué).
- Créer et gérer les droits de deux utilisateurs distincts : un compte administrateur ("alice") et un compte développeur.
- Configurer l'environnement de développement avec l'installation de **VS Code**.
- Rédiger une **procédure documentée reproductible** vérifiable par un tiers.

Résultats obtenus

- Machine Debian 12 entièrement configurée, légère et fonctionnelle sous VMWare.
- Séparation stricte des privilèges validée entre l'administratrice et le développeur.
- Environnement de travail prêt à l'emploi avec VS Code opérationnel.
- Procédure étape par étape testée et validée.

Environnement Debian optimisé (20 Go) avec séparation des rôles

Méthodes et technologies

VMWare Workstation, Debian 12 (ISO), VS Code, Bash / Terminal, Gestion utilisateurs Linux, Sudo / Droits

Compétences BUT exercées

- Administrer (système Debian et machines virtuelles)
- Gérer les droits des utilisateurs (Linux)
- Documenter (procédure reproductible)

Commentaires

Maîtrise de la virtualisation et de l'administration système sous Linux. Savoir configurer et gérer les droits d'un environnement de travail est essentiel pour tout poste de développeur ou DevOps.

Musicalertes — Création d'une base de données

Modélisation & SQL · Semestre 1 · Binôme

Cahier des charges

- Concevoir from scratch la BD de la plateforme **Musicalertes** pour un service d'alertes concerts/festivals.
- Réaliser 5 étapes : recensement des données, description des services, **MCD sous Win'Design**, génération du MLR et script SQL DDL commenté.
- Modéliser les entités : artistes, concerts, festivals, lieux, clients et alertes SMS/email.
- Fournir un script DML de remplissage.

Résultats obtenus

- MCD validé et MLR généré puis enrichi manuellement.
- Scripts SQL exécutables sans erreur, incluant contraintes CHECK et commentaires détaillés.
- Jeu de données créé : 3 festivals, 10 concerts, 5 artistes, 8 clients.

Scripts exécutés sans erreur — MCD approuvé

Méthodes et technologies

Win'Design, MCD / MLR, SQL DDL + DML, Clés étrangères, Contraintes CHECK, MySQL / MariaDB

Compétences BUT exercées

- Gérer les données (base de données)
- Modéliser (du MCD au SQL)
- Documenter (scripts commentés)

Commentaires

Assimilation du processus de modélisation de données (MCD/MLR) et de l'écriture de scripts SQL robustes. Une compétence clé, car la structuration des données est le cœur de toute application d'entreprise.

Installation de services réseaux (Nextcloud)

Réseaux & Systèmes · Semestre 2 · Binôme

Cahier des charges

- Mise en place d'une infrastructure réseau virtuelle complexe via **NEmu** avec deux sous-réseaux.
- Configuration d'une passerelle (routeur) pour assurer le routage inter-réseaux et l'accès Internet.
- Installation et configuration de **Nextcloud** pour le stockage et partage de fichiers, couplé à **OnlyOffice** pour l'édition collaborative.
- Déploiement de clients lourds Nextcloud sur des postes clients et production d'une **documentation technique (Wiki)**.

Résultats obtenus

- Infrastructure réseau déployée avec succès sous NEmu.
- Services serveurs fonctionnels et sécurisés (HTTPS, 2FA, gestion des accès).
- Documentation technique complète (Wiki format Markdown) détaillant toutes les étapes.

Réseau complet + Services serveur configurés — Wiki validé

Méthodes et technologies

NEmu, Debian 12, Nextcloud, OnlyOffice, Snap, HTTPS / TLS, Markdown

Compétences BUT exercées

- Administrer (réseaux et systèmes)
- Installer et configurer des services
- Documenter (Wiki en Markdown)

Commentaires

Mise en œuvre concrète d'une infrastructure réseau et de services de collaboration. Cette expérience me donne une vision globale du déploiement de services et de l'importance cruciale d'une documentation technique claire pour la maintenance.

Environnement réseau & administration système

Réseaux · Linux · Semestre 1 · Binôme

Cahier des charges

- Configurer un environnement réseau local complet en machines virtuelles.
- Mise en place de serveurs **DHCP, DNS et HTTP**, et configuration des interfaces réseau.
- Gérer les utilisateurs et groupes Linux, avec scripting **Bash** pour l'automatisation de tâches d'administration.
- Rédiger une procédure technique détaillée pour permettre la reproduction de l'environnement, incluant des diagrammes réseau.

Résultats obtenus

- Environnement réseau virtuellement déployé et entièrement opérationnel.
- Scripts d'automatisation Bash testés et validés en conditions réelles.
- Documentation complète livrée avec diagrammes réseau clairs et procédures étape par étape.

Environnement reproductible — documentation approuvée

Méthodes et technologies

Bash scripting, Ubuntu Linux, DHCP / DNS, Réseau LAN, VMWare, Droits Unix (chmod)

Compétences BUT exercées

- Administrer (réseau local)
- Automatiser (scripts Bash)
- Documenter (procédures et diagrammes réseau)

Commentaires

Compréhension concrète de l'architecture d'un réseau local (DHCP, DNS) et de l'automatisation via des scripts Bash. Ces bases réseaux sont cruciales pour le déploiement et le diagnostic d'infrastructures.

Les Mystères de BUTopia — App C# POO

Développement orienté objets · Semestre 2 · Trinôme

Cahier des charges

- Développer une application WinForms complète de gestion de **fiches de personnages pour un jeu de rôles fictif** (BUTopia).
- Implémenter des indicateurs numériques, des compétences critériées, un système d'épreuves avec jets de dés, un entraînement et une santé mentale évolutive.
- Assurer des livraisons de code en 4 stades distincts.
- Fournir un **dossier UML** complet (diagrammes de classes), des tests NUnit, et des compte-rendus de projet.

Résultats obtenus

- Application WinForms robuste intégrant création/édition de personnages, glisser-déposer et épreuves avec jets de dés.
- Système de gestion d'expérience implémenté et pleinement fonctionnel.
- Dossier UML fourni incluant un dictionnaire précis des responsabilités des classes.

App fonctionnelle — 100 % tests verts — UML livré

Méthodes et technologies

C# / .NET, WinForms, NUnit, UML Diagrammes, Propriétés C#, Enum + extensions, Drag & Drop, Git

Compétences BUT exercées

- Réaliser (programmation orientée objet en C#)
- Tester (NUnit)
- Collaborer (équipe de 3)
- Modéliser (UML)

Commentaires

Acquisition des concepts de la programmation orientée objet (POO) et de la modélisation UML. Savoir concevoir une architecture logicielle propre et modulaire est indispensable pour coder des projets complexes en équipe.

Retro-PC Dépannage — Refactorisation technique

Développement web moderne · Semestre 2 · Solo

Cahier des charges

- Refactoriser complètement le site « Retro-PC Dépannage » en utilisant des techniques web modernes sur 6 missions.
- Intégrer du HTML5 sémantique et des règles d'accessibilité strictes.
- Créer un **Design System CSS** avec variables :root et police variable Google Fonts.
- Développer un layout **CSS Grid + Flexbox** en approche Mobile-First, incluant des transitions et animations @keyframes.
- Obtenir une **validation W3C sans erreur** et fournir un rapport technique réflexif.

Résultats obtenus

- 3 pages HTML validées W3C sans aucune erreur résiduelle.
- Design system complet appliqué avec polices variables et animations fluides (fadeInUp, hover).
- Rapport réflexif exhaustif livré avec captures de la progression.

W3C vert — Design system complet — Rapport livré

Méthodes et technologies

HTML5 sémantique, CSS Grid, Flexbox, Variables CSS, @keyframes, Media queries, W3C Validator

Compétences BUT exercées

- Réaliser (développement web moderne)
- Concevoir un design system et soigner l'accessibilité
- Communiquer (rapport réflexif)

Commentaires

Maîtrise de l'intégration web moderne (Grid, Flexbox, accessibilité) et de la refactorisation de code. Cette rigueur front-end me permettra de créer des interfaces professionnelles, maintenables et évolutives.

Pixel War — Application JavaScript moderne

JavaScript · Fetch API · Canvas · Authentification · Semestre 2

Cahier des charges

- Développer une application JavaScript moderne de type « Pixel War » (canvas collaboratif), avec outillage professionnel.
- Configurer un environnement **Vite + npm** pour le bundling et la gestion des dépendances.
- Implémenter la manipulation du **Canvas HTML5** et des appels **Fetch API** vers un back-end JSON.
- Gérer l'**authentification JWT** via json-server-auth et structurer le code en modules ES6.

Résultats obtenus

- Application Pixel War complète et collaborative en temps réel.
- Auth JWT fonctionnelle, canvas interactif avec détection de clics et coloration persistée via API REST.
- Build Vite final optimisé et prêt pour la production.

App complète : auth + canvas + API REST — Build prod

Méthodes et technologies

JavaScript ES6+, Vite, npm, Canvas API, Fetch API, json-server-auth, JWT, Modules ES6, REST API

Compétences BUT exercées

- Réaliser (JavaScript moderne)
- Intégrer une API et une authentification
- Utiliser un outillage moderne (Vite, npm)

Commentaires

Prise en main de l'écosystème JavaScript professionnel (Vite, requêtes API REST, authentification JWT). Cette étape marque ma transition vers le développement web moderne, indispensable pour le développement fullstack.

Coloration de graphes — Application Java

Développement orienté objet, graphes et complexité · Semestre 2 · Binôme

Cahier des charges

- Développer une application Java capable de **représenter visuellement un graphe**, de manipuler ses sommets et ses arêtes, d'appliquer différents algorithmes de coloration et de comparer leurs résultats.
- Implémenter la classe **Graph** (interface IGraph) : ajout et suppression de nœuds et d'arêtes, requêtes de voisinage, propriétés visuelles (couleur, taille, position), à partir du code d'affichage fourni.
- Coder trois algorithmes de coloration : **Welsh-Powell**, **6-coloration** et **5-coloration** pour les graphes planaires.
- Concevoir des **tests unitaires JUnit**, puis **analyser la complexité** des méthodes et des algorithmes et mesurer leurs temps d'exécution.

Résultats obtenus

- Classe Graph complète, développée méthode par méthode avec un test écrit après chaque méthode.
- Trois algorithmes de coloration fonctionnels : aucun voisin de même couleur, au plus 6 puis 5 couleurs sur les graphes planaires (testé jusqu'à une grille de 400 nœuds).
- 23 tests unitaires réussis : 11 sur les méthodes utilitaires, 2 sur Welsh-Powell, 6 sur la 6-coloration et 4 sur la 5-coloration.
- Complexité de chaque méthode et de chaque algorithme calculée (Welsh-Powell et 6-coloration en $O(V \cdot E)$, 5-coloration en $O(V^2 \cdot E)$) et vérifiée par des mesures de temps sur 5 graphes créés par nos soins.

23 tests sur 23 réussis — 3 algorithmes de coloration complets — Analyse de complexité rédigée

Méthodes et technologies

Java, JUnit, UML (diagrammes de classes début et fin de projet), Javadoc, HashMap, Tri fusion, Récursivité, Graphes planaires, Complexité algorithmique, Python (mesures de performance)

Compétences BUT exercées

- Réaliser (développement orienté objet)
- Optimiser (algorithmes et complexité)
- Vérifier / Tester (JUnit)
- Modéliser (UML)
- Collaborer (binôme)

Commentaires

Découverte de la théorie des graphes et de l'analyse de complexité sur un vrai projet Java. J'ai compris l'impact du choix d'une structure de données : sans liste d'adjacence, chaque requête de voisinage coûte O(E), ce qui limite les performances de tous les algorithmes. Un réflexe utile pour écrire du code efficace.

Blogue statique — Script Bash

Scripting Bash · Markdown · Pandoc

Cahier des charges

- Écrire un script **Bash** de gestion de blogue ou de site statique, appelé avec une commande et un paramètre optionnel (./blog.sh COMMANDE [PARAMETRE]).
- Gérer des pages écrites en **Markdown** : **éditer**, **supprimer** (avec confirmation) et **lister** les documents.
- **Construire** le site avec **Pandoc** : sommaire (index.md), une page HTML par document et un PDF unique (blog.pdf).
- **Visualiser** le résultat avec xdg-open et fournir une commande **usage** qui affiche la documentation du script.

Résultats obtenus

- Script complet : cinq commandes (éditer, supprimer, lister, construire, visualiser) et la commande usage.
- Choix automatique de l'éditeur : variable d'environnement $EDITOR, puis /usr/bin/editor, puis recherche d'un éditeur présent sur la machine.
- Garde-fous : demande de confirmation avant toute suppression et message clair « Il n'y a pas de fichier » quand le répertoire est vide.
- Construction du site : sommaire généré avec liens HTML et mention « Contenu actualisé le [date] par [utilisateur] », fichiers HTML dans web/ et PDF unique blog.pdf. Trace d'exécution fournie pour chaque commande.

Script fonctionnel — 5 commandes + usage — Site HTML et PDF généré avec Pandoc

Méthodes et technologies

Bash, Pandoc, Markdown, xdg-open, Variables d'environnement ($EDITOR), Conditions Bash, Génération HTML / PDF

Compétences BUT exercées

- Automatiser (Bash scripting)
- Réaliser (outil en ligne de commande)
- Documenter (usage, traces d'exécution)

Commentaires

Les principales difficultés ont été les conditions et la syntaxe Bash, la commande build (enchaîner les étapes de génération avec Pandoc) et la gestion des erreurs. Ce projet m'a appris à automatiser une chaîne de publication complète en ligne de commande et à rendre un script plus sûr : confirmation, messages clairs et documentation intégrée.

Risk prevention in Nouvelle-Aquitaine

Exploitation de base de données & gestion de projet · Semestre 2 · Équipe de 6

Cahier des charges

- Concevoir une application de **visualisation et d'analyse des risques naturels** en Nouvelle-Aquitaine (inondations, incendies de forêt, vagues de chaleur...) à destination des habitants et du grand public, en s'inscrivant dans le cadre du défi national OpenDataUniversity (data.gouv.fr).
- S'appuyer sur **trois sources Open Data** : la base Gaspar (catastrophes naturelles et plans de prévention des risques), les données météo quotidiennes de Météo-France et les mesures de qualité de l'air d'Atmo Nouvelle-Aquitaine.
- Rédiger le **dossier de spécifications** : lexique, fonctions priorisées, contraintes, structure du logiciel, modèle conceptuel de la base de données et caractérisation de l'interface.
- Mener le projet en **équipe de 6** en adaptant localement les consignes et les objectifs de ce challenge public

Contraintes

- Utiliser des données historiques sur les risques naturels, la qualité de l'air et le climat, puis les **nettoyer** pour ne garder que les plus pertinentes.
- Réaliser une **analyse statistique** avec régression linéaire partielle et histogrammes.
- Intégrer les données en fixe (sans requêtes en ligne) dans une interface graphique.

Architecture et données

- Architecture en **trois couches** : interface graphique en **C#** (.NET 8.0, Visual Studio), moteur de traitement en **Python** (chargement, nettoyage, analyse statistique, graphiques) et **base de données** regroupant les trois sources.
- **Volume de données important** : de l'ordre de 4 millions de données traitées en Python avant et pendant leur intégration dans la base.
- Bibliothèques Python utilisées pour l'analyse : Pandas (traitement), Matplotlib et Seaborn (graphiques), GeoPandas (données géographiques).

Résultats obtenus

- Dossier de spécifications rédigé à 6 : lexique, fonctions priorisées (doit avoir, devrait avoir, peut avoir, n'aura pas), contraintes, structure du logiciel, modèle conceptuel de la base de données et caractérisation de l'interface.
- Base de données regroupant les trois sources Open Data (Gaspar, Météo-France, Atmo Nouvelle-Aquitaine), avec environ 4 millions de données traitées en Python.
- Application en trois couches (interface C#, moteur Python, base de données) permettant de croiser risques naturels, météo et qualité de l'air.

Application en 3 couches — 3 sources Open Data — environ 4 millions de données traitées en Python

Méthodes et technologies

C# (.NET 8.0), Visual Studio, Python, Pandas, Matplotlib, Seaborn, GeoPandas, SQL, SQLite, Régression linéaire, Données Open Data

Compétences BUT exercées

- Gérer les données (base de données et Open Data)
- Réaliser (interface C# et traitement Python)
- Collaborer (équipe de 6)
- Documenter (dossier de spécifications)

Commentaires

Projet de grande ampleur : une équipe de 6, trois technologies (C#, Python, SQL) et un volume de données de l'ordre de 4 millions. J'ai appris à nettoyer et traiter de gros jeux de données avec Python, à relier plusieurs langages dans une même application et à travailler avec un cahier des charges priorisé pour répartir le travail.